Introduction

Most Python developers write `obj.attr` thousands of times without ever asking what actually happens when that line executes. It looks like a dictionary lookup. It isn't. Python runs a small, deterministic search across the object's type and the object's instance state, and at any point in that search, a class can install a hook that intercepts the whole process. That hook is called a descriptor, and it's the exact mechanism behind `@property`, Django model fields, `functools.cached_property`, and a good chunk of what makes ORMs feel like magic.

This matters for two overlapping reasons. First, it shows up constantly in interviews — not usually as "implement a descriptor" cold, but as "why did this attribute assignment fail" or "explain what `@property` actually does under the hood." Second, it's one of those topics that, once it clicks, makes a dozen previously-mysterious framework behaviors suddenly obvious. You stop treating `__slots__`, `super()`, and custom metaclasses as separate tricks and start seeing them as four expressions of the same underlying protocol.

By the end of this article you'll have built a working validating descriptor, understood exactly why `__slots__` can silently fail to save memory, know how Python computes a single deterministic method resolution order when multiple parents define the same method, and written a small metaclass that auto-registers every subclass the moment it's defined.

Problem Overview

Here's the puzzle that kicks this off: take two classes that look nearly identical. Create an instance of the first one and assign it a brand-new attribute it was never given in `__init__`. That works — Python happily adds it. Do the exact same thing to an instance of the second class, and Python raises an `AttributeError`, even though nothing about the syntax changed.

The instinct is to assume a bug. It isn't one. The second class made a deliberate choice — probably via `__slots__` — to opt out of the flexible, per-instance dictionary that normally backs every Python object. Understanding why requires understanding what `obj.attr` is actually doing, which is where descriptors, `__dict__`, and the MRO all come in.

We'll build this up piece by piece: first the attribute-lookup protocol itself, then the specific mechanism (`__slots__`) that explains the two-classes puzzle, then MRO for the "which parent method wins" question, and finally metaclasses, which control how classes themselves get built.

Example

Start with the attribute puzzle concretely:

```
class Flexible:
    def __init__(self, value):
        self.value = value

class Rigid:
    __slots__ = ("value",)
    def __init__(self, value):
        self.value = value

f = Flexible(1)
f.extra = "this works fine"

r = Rigid(1)
r.extra = "this raises AttributeError"
```

`f.extra = ...` succeeds because `Flexible` instances carry a `__dict__` — an ordinary dictionary that Python is free to add new keys to at any time. `r.extra = ...` fails because `Rigid` declared `__slots__ = ("value",)`, which tells Python: don't allocate a `__dict__` for this class at all, only allocate fixed storage for the named slots. There is no dictionary left for `extra` to land in, so the assignment has nowhere to go and Python raises immediately.

That single difference — dict or no dict — is the entire explanation for the opening puzzle. But it only makes sense once you know what `__dict__` is doing in the lookup process to begin with.

Intuition

Think of `obj.attr` not as a lookup but as a small search with an order of priority, and think of that search as passing through a front desk before it reaches the object's own storage. Some class-level attributes are ordinary values sitting there. Others are special objects — descriptors — that implement `__get__` and, optionally, `__set__`. When Python's search finds one of those, it doesn't hand you the descriptor object itself; it calls the descriptor's method and gives you whatever that method returns (or does).

The practical value of this shows up the moment you need validation on an attribute. Say you have an `Account` class with a `balance`, and balance must never go negative. Without a descriptor, you'd scatter `if value < 0: raise ValueError(...)` at every point in your codebase that sets `balance`, and it only takes one missed spot for a negative balance to slip through with no error at all.

A descriptor closes that gap once, in one place, because every single assignment to `balance` — no matter where in the codebase it happens — is forced to route through the same `__set__` method.

This is the intuition an experienced engineer reaches for: any time "this rule must hold no matter how the attribute is touched" comes up, that's a signal to centralize the check at the attribute-access layer instead of relying on every caller to remember it.

Brute Force Solution

The brute-force version of "enforce this invariant everywhere" is exactly what was just described: manual guards repeated at every call site.

Idea: Every place in the code that assigns to `balance` includes an explicit check before the assignment.

```
if new_balance < 0:
    raise ValueError("balance cannot be negative")
account.balance = new_balance
```

Advantages: - No new concepts required — plain conditionals anyone can read. - Works fine in a script with one or two assignment sites.

Disadvantages: - Not enforced by the language — it's enforced by discipline, and discipline doesn't scale across a team or a growing codebase. - Every new call site is a chance to forget the check, and the failure mode is silent (a bad value just gets stored). - Duplicated logic means a change to the rule (say, adding a maximum limit) has to be found and updated everywhere.

Time complexity: O(1) per assignment, same as the descriptor version — the cost here isn't runtime, it's maintainability. **Space complexity:** O(1) — no extra structures, which is also why it's tempting to skip building anything more robust.

Optimal Solution

The optimal approach moves the check into a descriptor, so the language itself enforces it instead of relying on every call site to remember.

A data descriptor is any object that defines `__get__` and `__set__` and is placed as a **class attribute**. When Python resolves `instance.attr`, it checks the type first; if what it finds there is a data descriptor, that descriptor's methods run instead of touching the instance's `__dict__` directly — even before the instance dict is consulted. That priority order (data descriptor on the type overrides

instance `__dict__`) is what makes this reliable: there's no way to route around it by poking at the instance dictionary directly.

Step	What happens
`account.balance` (read)	Python finds `balance` is a descriptor on the class → calls its `__get__` → returns the stored value
`account.balance = -5` (write)	Python finds the descriptor's `__set__` → validation runs → raises before anything is stored
`account.balance = 100` (write)	`__set__` validation passes → value is stored in the descriptor's own backing storage

Because the descriptor lives on the class and every instance shares it, there's exactly one place the rule can be defined and exactly one place it needs to change.

Python Code

```
class NonNegative:
    """Descriptor enforcing that an attribute never goes negative."""

    def __set_name__(self, owner, name):
        self._name = "_" + name

    def __get__(self, instance, owner):
        if instance is None:
            return self
        return getattr(instance, self._name)

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError(f"{self._name.lstrip('_')} cannot be negative")
        setattr(instance, self._name, value)


class Account:
    balance = NonNegative()

    def __init__(self, balance):
        self.balance = balance
```

Code Walkthrough

- `__set_name__(self, owner, name)` is called automatically by Python once, when the class body finishes executing, and tells the descriptor what name it was assigned to (`balance`). It's

used here to pick a private backing-attribute name (`_balance`) so the descriptor doesn't collide with itself.

- `__get__(self, instance, owner)` runs on every read of `account.balance` . The `instance is None` check handles the case where the descriptor is accessed on the class itself (`Account.balance`) rather than an instance; returning `self` there is the standard convention.
- `__set__(self, instance, value)` runs on every write, including the one inside `__init__` . It validates before storing anything, which is the entire point — there is no code path that stores a value without passing through this check.
- `Account.balance = NonNegative()` replaces what would otherwise be a plain instance attribute with a class-level descriptor. This is the swap that routes every future `self.balance = x` through `__set__` instead of writing straight into `self.__dict__` .

Dry Run

```
a = Account(50)
```

`__init__` runs `self.balance = 50` . Python sees `balance` is a data descriptor on `Account` , so it calls `NonNegative.__set__(descriptor, a, 50)` . `50 < 0` is false, so it stores `50` under `a._balance` .

```
print(a.balance)
```

Python sees the descriptor again, calls `__get__(descriptor, a, Account)` , which returns `getattr(a, "_balance")` → `50` .

```
a.balance = -10
```

`__set__` runs again, `-10 < 0` is true, and a `ValueError` is raised before `_balance` is ever touched — the object's state stays exactly as it was.

Complexity Analysis

Time: O(1) for both read and write — the descriptor adds one extra method call over a plain attribute access, no loops or scans involved. **Space:** O(1) additional per instance — one extra attribute (`_balance`) instead of `balance` , no growth with input size. These are correct because the descriptor

doesn't introduce any data-dependent branching or storage; it's a fixed, constant amount of indirection layered on top of ordinary attribute storage.

Alternative Solutions

`@property` is the built-in shorthand for exactly this pattern when the logic is specific to one class:

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("balance cannot be negative")
        self._balance = value
```

`property` is itself implemented as a descriptor, so this isn't really a different mechanism — it's the same protocol with less boilerplate. The custom descriptor class is worth writing instead when the same validation rule needs to be reused across multiple unrelated classes and attributes (`NonNegative()` can be dropped onto any class), whereas `property` is naturally a one-off per class.

Edge Cases

- **Assigning during `__init__` before validation exists:** if `__set_name__` hasn't run yet (rare, but possible with dynamic class construction), `self._name` may not be set — always define descriptors at class-definition time, not dynamically after the fact.
- **Zero as a boundary value:** `0 < 0` is `False`, so zero is correctly accepted — worth testing explicitly since off-by-one mistakes here are common.
- **Subclassing without re-declaring `__slots__`:** every class in an inheritance chain must declare `__slots__` for the memory savings to hold; one class without it reintroduces a `__dict__` for the whole chain, silently.
- **Multiple inheritance with same-named methods:** resolved by MRO, not by declaration order guesswork — always verify with `ClassName.__mro__` rather than assuming.
- **Accessing the descriptor via the class, not an instance:** must be handled (`instance is None` check) or `ClassName.balance` will crash instead of returning the descriptor object.

Common Mistakes

1. **Storing the value directly on `self.balance` inside the descriptor's `__set__`.** This creates infinite recursion, since setting `self.balance` re-triggers the descriptor. Always store under a different name (`_balance`).
2. **Forgetting `__set_name__` and hardcoding the backing attribute name.** Breaks the moment the descriptor is reused for a second attribute on the same class.
3. **Assuming `__slots__` savings apply automatically to subclasses.** If any parent in the chain omits `__slots__`, the subclass gets a `__dict__` anyway, with no warning.
4. **Confusing non-data descriptors (only `__get__`) with data descriptors (`__get__` + `__set__`).** Only data descriptors take priority over instance `__dict__`; a non-data descriptor can be silently shadowed by an instance attribute of the same name.
5. **Assuming `super()` looks at "the parent class."** It walks the MRO, which in multiple inheritance is not necessarily the direct parent — it depends on the full hierarchy.

Interview Questions

- What's the difference between a data descriptor and a non-data descriptor, and why does the distinction affect lookup priority?
- Why does `__slots__` prevent adding new attributes, and what happens if a subclass forgets to declare it?
- Two parent classes define the same method — which one runs, and how would you verify it without guessing?
- What does a metaclass control that a regular class doesn't?
- How is `@property` implemented internally?

Similar Problems

- **LeetCode-style "design" problems (e.g., Design HashMap, LRU Cache):** share the same underlying skill of exposing a clean interface (`obj[key]`, `obj.attr`) while controlling what happens underneath it — descriptors are Python's built-in version of that same idea for attribute access.
- **Implementing a custom context manager (`__enter__` / `__exit__`):** another case where Python calls specific dunder methods in a fixed protocol, reinforcing the "protocol, not magic" mental model.
- **Singleton pattern via metaclass:** a natural follow-up exercise once `__new__` on a metaclass is understood — restricting instantiation instead of registering classes.

Key Takeaways

`obj.attr` is a search, not a direct grab — it checks the type first, then the instance, and descriptors on the type can intercept that search entirely. `__slots__` removes the instance `__dict__` that flexible attribute assignment depends on, and that removal must be opted into by every class in an inheritance chain to actually save memory. When multiple parents define the same method, Python's MRO — not guesswork — decides which one runs, and `super()` always follows that same order. A metaclass controls how a class itself gets built, which is what lets patterns like auto-registration happen without any class remembering to call a registration function. All four of these are expressions of one idea: attribute and class construction in Python are protocols you can hook into, not fixed rules.

Watch the Video

This entire walkthrough — including the live two-class attribute error, the descriptor build, and the auto-registering metaclass — is demonstrated step by step in the video. Watch it here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a daily series that takes developer topics — Python internals, algorithms, interview-style problems, and real framework mechanics — and breaks them down from first principles instead of treating them as memorized tricks. New lessons land every day on the same channel.

Call To Action

If this made Python's object model click a little more, give the video a like on YouTube and subscribe to catch new lessons as they land. For the written breakdowns and code samples like this one, subscribe to the Substack. Drop your MRO guess in the comments before checking it against the video, and share this with anyone still treating `@property` as unexplainable magic.

Quick Review

Descriptor: define __get__/__set__ on a class attribute. Trigger?

Instance dict lookup finds an attribute defined on the class that has __get__/__set__ — Python calls those instead of returning it plain.

Attribute lookup cost: obj.attr time complexity?

O(MRO length) worst case — walks instance dict, then each class in MRO until found or AttributeError.

___slots___ space effect vs normal ___dict___?

Removes per-instance ___dict___, saving ~memory per object; O(1) fixed attribute slots instead of a dynamic dict.

Trickiest bug: ___slots___ subclass with no ___slots___ itself?

Child class without ___slots___ silently regains a ___dict___, cancelling the parent's memory savings — easy to miss.

Data descriptor vs non-data descriptor: what changes lookup order?

Data descriptor (has ___set___) always wins over instance dict; non-data descriptor (only ___get___) loses to instance dict if attr present there.

Follow-up: why does a class behave like 'just an object'?

Classes are instances of a metaclass (type); metaclass's ___call___/___new___ controls class creation, same as instances of a normal class.
